

Inteligência Artificial

Trabalho prático: Detenção automática da ocupação de um espaço interior



Aluno: Cassiano Videira, 37882

Valter Pino, 38370

Liliana Paquete, 43720

Docente: Paulo Duarte Ferreira Gouveia

Licenciatura em Engenharia Informática
2021-2022

Resumo

O presente trabalho consiste na exploração de 3 modelos de “Machine Learning” (ML), Regressão Logística, Os K Vizinhos Mais Próximos e Árvore de Decisão, por meio de um Dataset que contem as respectivas respostas na variável Ocupado (Resposta), e por fim escolher o modelo com maior score para fazer a detenção automática de Ocupado e Não ocupado, de um Dataset sem a variável Ocupado (Resposta). O presente relatório será focado apenas no modelo com melhor desempenho.

Desenvolvimento

1. Implementação da solução

1.2 Importar o Pandas, Numpy, e o Seaborn, e importar o Dataset:

Na imagem a baixo é possível observar a importação do Dataset Pandas, Numpy, Seaborn, logo em seguida é possível observar na variável data_file a importação do dataset através dos pandas ”pd”.

Figura 1 Importação do Pandas, Numpy, Seaborn e Dataset

```
import pandas as pd #Importar o pandas para lidar com os datasets
import numpy as np #Importar o numpy para lidar com arrays
import seaborn as sns #Importar o Seaborn para criar o Grafico

data_file = pd.read_csv('dataset.csv') #Importando o dataset

data_file
```

1.3 Eliminar Linhas duplicadas e Nans:

Na linha a baixo é possível observar o comando que foi usado para eliminar os Nans, e o comando que foi usado para eliminar as linhas duplicadas a qual se mantem sempre a primeira linha elimina-se a segunda.

Figura 2 Eliminação de Nans e Linhas duplicadas

```
data_file.dropna(inplace=True)#Eliminar Linhas duplicadas
data_file.drop_duplicates(keep='first',inplace=True)#Eliminar Linhas duplicadas mantendo sempre a primeira
data_file.count()
```

1.4 Relação das variáveis Predictoras com a variável Alvo:

Na imagem a baixo é possível ver o comando que foi usado para podermos observar qual a relação que as variáveis independentes tem com a variável alvo.

Figura 3 Relação das variáveis independentes com a variável alvo

```
print(data_file.corr().ocupado.abs().sort_values())#Ver a relação que as variaveis preditivas tem com a variavel alvo
```

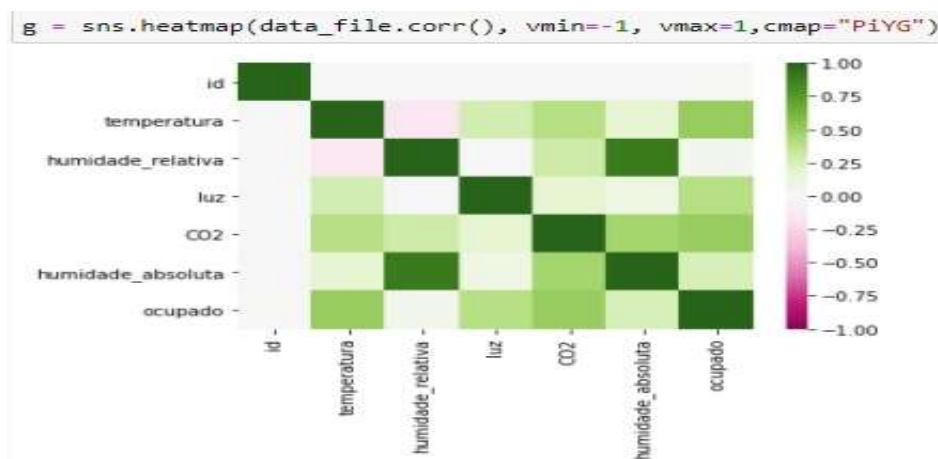
id	0.000311
humidade_relativa	0.056890
humidade_absoluta	0.262842
luz	0.404335
CO2	0.502164
temperatura	0.511779
ocupado	1.000000

Name: ocupado, dtype: float64

1.5 Gráfico de Heatmap

Na imagem a baixo é possível observar o gráfico heatmap que nos ajuda também a perceber a relação das variáveis independentes com a variável alvo.

Figura 4 Gráfico Heatmap



1.6 Importação do Train_test_split e particionando o dataset

Na imagem a baixo é possível observar a importação do train_test_split que permite-nos particionar o dataset. Logo a baixo do import é possível observar a criação das partições, a qual foi dado 25% do dataset para os dados de teste, foi dado para o Random_state 1, e foi usado o stratify com a variável alvo de forma

a mantendo o equilíbrio entre os valores ocupados e não ocupados nas partições criadas.

Figura 5 Importação do `train_test_split` e particionando o dataset

```
from sklearn.model_selection import train_test_split

Xtreino,Xteste,ytreino,yteste =train_test_split(data_file.drop("ocupado",axis=1),data_file[["ocupado"]],test_size=0.25, random_state=1,stratify=data_file.ocupado)
```

1.7 Valores de Ocupado e Não ocupado na variável

Na imagem a baixo é possível observar os valores disponíveis de Ocupado e Não ocupado na variável `ytreino`.

Figura 6 Valores de Ocupado e Não ocupado na variável `ytreino`

```
ytreino.value_counts()#Ver o numero de ocupados e não ocupados da variavel Ytreino
```

ocupado	
0.0	8066
1.0	2428

dtype: int64

2. Arvore de decisão

2.1 Fazendo a importação da Arvore de decisão

Na imagem a baixo é possível observar a importação da Arvore de decisão.

Figura 7 Importação da Arvore de decisão

```
from sklearn.tree import DecisionTreeClassifier #Modelo Arvores de Decisão
modelo_DecisionTree = DecisionTreeClassifier()
```

2.2 Treinando o modelo com os dados de treino

Na imagem a baixo é possível ver o modelo a treinar com os dados de treino (`Xtreino`, e `ytreino`).

Figura 8 Treinando o modelo

```
modelo_DecisionTree.fit(Xtreino,ytreino)#Treinando o modelo
```

DecisionTreeClassifier()

2.3 Obtendo o score do modelo com os dados de teste

Na imagem a baixo é possível observar o score obtido com os dados de teste.

Figura 9 Obtendo o Score do modelo

```
modelo_DecisionTree.score(X=Xteste, y=yteste)#score do modelo treinado  
0.9156901971991998
```

2.4 Importando o GridSearchCV e criando a grelha de valores

Na imagem a baixo é possível observar a importação do GridSearchCV(Tuning) que serviu para procura do melhor modelo, e foram usados os hiperparâmetros: Profundidade máxima(max_depth), Amostras mínimas divididas(min_samples_split), Minimo de amostras de folha (min_samples_leaf).

Figura 10 Importando o GridSearchCV

```
from sklearn.model_selection import GridSearchCV#Importando o GridSearchCV  
  
#Criando a Grelha de valores para o GridSearchCV  
grelha_valores={'criterion':['gini','entropy'],'max_depth':[2,3,4,5,10,20,30], "m  
,"min_samples_split":range(1,20),"min_samples_leaf":range(1,15)}
```

2.5 Treinando o modelo Tuning

Na imagem a baixo é possível observar a procura do melhor modelo baseando-se no modelo já criado com a Arvore de decisão e com os hiperparâmetros definidos anteriormente na grelha de valores, foi também fornecido 5 partições para a CV.

Figura 11 Criação do modelo tuning

```
modelo_DecisionTree = DecisionTreeClassifier()  
procura_modelo = GridSearchCV(modelo_DecisionTree,param_grid=grelha_valores,cv=5  
procura_modelo.fit(X=Xtreino, y=ytreino)#Treinando o modelo com os novos paramet  
Fitting 5 folds for each of 3724 candidates, totalling 18620 fits
```

```
,param_grid=grelha_valores,cv=5,verbose=1,n_jobs=-1)
# o modelo com os novos parametros do GridSearchCV
```

2.6 Melhores parâmetros obtidos no Tuning e o respetivo score

Na imagem a seguir é possível observar os melhores parâmetros obtidos no Tuning e o respetivo score.

Figura 12 Melhores parâmetros e respetivo score do tuning

```
procura_modelo.best_params_#Apresentando os melhores parametros do novo modelo
{'criterion': 'entropy',
 'max_depth': 20,
 'min_samples_leaf': 11,
 'min_samples_split': 7}

modelo_otimo=procura_modelo.best_estimator_

modelo_otimo.score(X=Xteste, y=yteste)#Apresentando o score do novo modelo
0.9334095455844527
```

2.7 Prevendo os valores da partição Xteste

Na imagem é possível observar o modelo_otimo(tuning) a prever os valores de ocupação com as variáveis preditoras presentes na partição Xteste.

Figura 13 Prevendo os valores de ocupação para a partição Xteste

```
resultado_arvoreDecisão=modelo_otimo.predict(Xteste)#Prevendo os valores de ocu
```

3. Métrica F1

3.1 Importação da métrica F1 e o respetivo score

Na imagem a baixo é possível observar a importação da métrica f1_score e logo em seguida a respetiva implementação que foi usado os valores previstos pelo modelo_otimo(tuning), e os valores reais da variável yteste.

Figura 14 Import e implementação da metrica F1

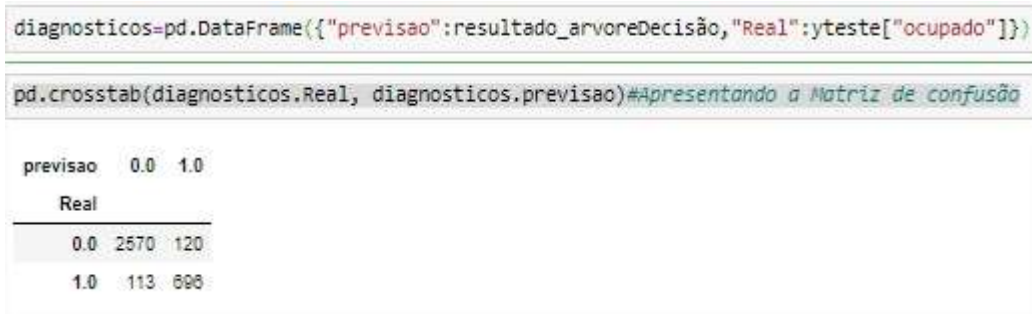
```
from sklearn.metrics import f1_score #Importando a metrica f1_score

f1_score(y_true=yteste, y_pred=resultado_arvoreDecisão, average=None)#Métrica F1
array([0.9560685 , 0.85485855])
```


4. Matriz de confusão

Na imagem a baixo é possível observar a criação da matriz de confusão que serve para avaliar o grau de Precisão e Sensibilidade do nosso modelo ótimo.

Figura 15 Criação da matriz de confusão



5. Prevendo os dados para o dataset não classificados

5.1 Importar o dataset não classificados

Na imagem a baixo é possível observar a importação do modelo não classificados.

Figura 16 Importação dos dados não classificados

```
Não_Classificados = pd.read_csv('dataset_naoclassificado.csv') #Importando nao c
```

5.2 Previsão com os dados não classificados

Na próxima figura é possível observar o modelo_otimo a prever a ocupação com os dados do dataset não classificados, e é também criada uma tabela com id e os dados de ocupação previstos.

Figura 17 Previsão dos dados não classificados e criação de tabela com os dados

```
Previsão_Não_Classificados=modelo_otimo.predict(Não_Classificados)#Passando a lista a classificar  
Resultado_Final=pd.DataFrame({'id_registro':Não_Classificados.id,'ocupacao':Previsão_Não_Classificados})# Cr
```

Na imagem a baixo é possível observar a tabela com o id e os valores de ocupação previstos.

Figura 18 Tabela com id e os dados de ocupação previstos para o dataset não classificados



id_registro	ocupacao
0	14001
1	14002
2	14003
3	14004
4	14005
...	...
5555	19556
5556	19557
5557	19558
5558	19559
5559	19560

5.3 Fazendo o export dos dados previstos para o dataset não classificado

Na imagem a baixo é possível ver o export dos dados da tabela anterior com o id e os valores previstos da ocupação para o dataset não classificado.

Figura 19 Export dos dados de ocupação previstos para a tabela não classificados

```
Resultado.to_csv(r'classificacao_estimada.csv',index = False, header=True)
```

6. Conclusão

No presente trabalho podemos aprofundar o estudo nos 3 modelos, Regressão Logística, Os K Vizinhos Mais Próximos e Árvore de Decisão, com o dataset. O modelo que conseguimos obter maior score foi o de Árvore de Decisão. Logo em seguida foi usado o GridSearchCV que foi possível aumentar o desempenho do modelo e após vários testes com o modelo, usamos a métrica F1 e a Matriz de Confusão que nos ajudou a analisar o desempenho do modelo. Por fim usamos o dataset Não classificados e com o modelo ótimo obtido pelo GridSearchCV, foi possível prever os dados sendo assim possível extraír os dados previstos pelo modelo em formato CSV. Com isto podemos concluir que a Inteligência Artificial tem vindo a ser uma área muito importante e requisitada para a evolução da humanidade, pois ajuda-nos a encontrar soluções para determinados problemas de forma automatizada.